

## Sesión de Aprendizaje: Introducción a MySQL y SQL

**Objetivo de la Sesión:** Aprender a utilizar MySQL para realizar consultas, manipular datos y aplicar funciones, a través de una serie de conceptos y comandos básicos.

### 1. Introducción a MySQL (15 min)

- **Teoría:**
  - **MySQL** es un sistema de gestión de bases de datos relacional (RDBMS) que utiliza el lenguaje SQL (Structured Query Language) para gestionar y manipular datos.
  - Es ampliamente utilizado por su robustez, facilidad de uso y capacidad de manejar grandes volúmenes de datos.
  - **SQL** es un estándar de la industria para trabajar con bases de datos. Permite realizar operaciones de creación, consulta, actualización y eliminación de datos.

### 2. Comandos Básicos de SQL (45 min)

- **SELECT:** Permite extraer datos de una base de datos.
  - **Teoría:** La cláusula **SELECT** define qué columnas se van a mostrar en los resultados.
  - Ejemplo: `SELECT * FROM empleados;`
- **WHERE:** Filtra los resultados basándose en condiciones específicas.
  - **Teoría:** Se utiliza para especificar criterios que deben cumplirse para que una fila sea incluida en el resultado.
  - Ejemplo: `SELECT * FROM empleados WHERE salario > 30000;`
- **AND, OR, NOT:** Permiten combinar múltiples condiciones.
  - **Teoría:** **AND** requiere que ambas condiciones sean verdaderas; **OR** requiere que al menos una sea verdadera; **NOT** invierte el valor lógico de la condición.
  - Ejemplo: `SELECT * FROM empleados WHERE departamento = 'Ventas' AND salario > 30000;`
- **ORDER BY:** Organiza los resultados en un orden específico.
  - **Teoría:** Se puede ordenar de forma ascendente (ASC) o descendente (DESC).
  - Ejemplo: `SELECT * FROM empleados ORDER BY salario DESC;`

### 3. Manipulación de Datos (60 min)

- **INSERT INTO:** Agrega nuevos registros a una tabla.
  - **Teoría:** Permite insertar uno o varios registros a la vez.
  - Ejemplo: `INSERT INTO empleados (nombre, salario) VALUES ('Juan', 25000);`
- **NULL Values:** Manejo de valores nulos.
  - **Teoría:** Un valor nulo indica la ausencia de un dato. Es importante saber cómo manejarlos en consultas.
  - Ejemplo: `SELECT * FROM empleados WHERE bonus IS NULL;`

- **UPDATE:** Modifica registros existentes.
    - **Teoría:** Permite cambiar uno o más valores de un registro existente en una tabla.
    - Ejemplo: `UPDATE empleados SET salario = 30000 WHERE nombre = 'Juan';`
  - **DELETE:** Elimina registros.
    - **Teoría:** Se utiliza para remover uno o varios registros de una tabla.
    - Ejemplo: `DELETE FROM empleados WHERE nombre = 'Juan';`
4. **Funciones Agregadas y Filtrado (30 min)**
- **MIN y MAX:** Obtienen el valor mínimo y máximo de una columna.
    - **Teoría:** Estas funciones son útiles para analizar datos numéricos.
    - Ejemplo: `SELECT MIN(salario), MAX(salario) FROM empleados;`
  - **COUNT, AVG, SUM:** Funciones de agregación que permiten obtener recuentos, promedios y sumas.
    - **Teoría:** Utilizadas para resumir información de múltiples filas.
    - Ejemplo: `SELECT COUNT(*), AVG(salario), SUM(salario) FROM empleados;`
  - **LIKE y Wildcards:** Realiza búsquedas de patrones en cadenas de texto.
    - **Teoría:** `LIKE` se utiliza junto con caracteres comodín (`%` y `_`) para buscar coincidencias.
    - Ejemplo: `SELECT * FROM empleados WHERE nombre LIKE 'J%';`
  - **IN y BETWEEN:** Permiten filtrar datos basados en un conjunto de valores o en un rango.
    - **Teoría:** `IN` verifica si un valor está en un conjunto especificado; `BETWEEN` verifica si un valor está dentro de un rango.
    - Ejemplo: `SELECT * FROM empleados WHERE salario BETWEEN 20000 AND 30000;`
5. **Alias y Agrupaciones (30 min)**
- **Alias:** Renombra columnas o tablas para mejorar la legibilidad.
    - **Teoría:** Se utiliza la palabra clave `AS` para crear un alias.
    - Ejemplo: `SELECT nombre AS NombreEmpleado FROM empleados;`
  - **GROUP BY:** Agrupa filas que tienen los mismos valores en columnas especificadas.
    - **Teoría:** Es útil para aplicar funciones de agregación a grupos de datos.
    - Ejemplo: `SELECT departamento, AVG(salario) FROM empleados GROUP BY departamento;`
  - **HAVING:** Filtra grupos basados en condiciones de agregación.
    - **Teoría:** Se utiliza después de `GROUP BY` para filtrar los resultados agregados.

- Ejemplo: `SELECT departamento, COUNT(*) FROM empleados GROUP BY departamento HAVING COUNT(*) > 5;`

## 6. Joins (30 min)

- **INNER JOIN:** Combina filas de dos o más tablas cuando hay coincidencias.
  - **Teoría:** Se utiliza para combinar datos de tablas relacionadas.
  - Ejemplo: `SELECT empleados.nombre, departamentos.nombre FROM empleados INNER JOIN departamentos ON empleados.depto_id = departamentos.id;`
- **LEFT JOIN, RIGHT JOIN, CROSS JOIN:** Otras maneras de combinar datos según la relación entre tablas.
  - **Teoría:** `LEFT JOIN` incluye todos los registros de la tabla izquierda y los coincidentes de la derecha; `RIGHT JOIN` lo opuesto; `CROSS JOIN` produce el producto cartesiano.
- **Self Join:** Une una tabla consigo misma para comparar registros.
  - **Teoría:** Útil para comparar filas dentro de la misma tabla.
- **UNION:** Combina los resultados de dos o más consultas.
  - **Teoría:** Las consultas deben tener el mismo número de columnas y tipos de datos compatibles.

## 7. Condiciones Avanzadas (15 min)

- **EXISTS, ANY, ALL:** Comprobaciones en subconsultas.
  - **Teoría:** `EXISTS` devuelve verdadero si una subconsulta devuelve al menos una fila; `ANY` y `ALL` se utilizan para comparar un valor con un conjunto de resultados.
- **INSERT SELECT:** Inserta datos a partir de otra consulta.
  - **Teoría:** Permite insertar múltiples registros de una consulta en una tabla.
- **CASE:** Permite ejecutar lógica condicional dentro de una consulta.
  - **Teoría:** Funciona como un `if` en programación.
- **NULL Functions:** Funciones para manejar valores nulos, como `IFNULL` y `COALESCE`.
  - **Teoría:** Utilizadas para sustituir valores nulos por valores predeterminados.

## 8. Cierre y Comentarios (15 min)

- Revisión de comentarios en SQL: Cómo documentar el código.
    - **Teoría:** Los comentarios ayudan a mantener el código legible y comprensible.
  - Revisión de operadores lógicos y aritméticos.
  - Preguntas y respuestas.
-